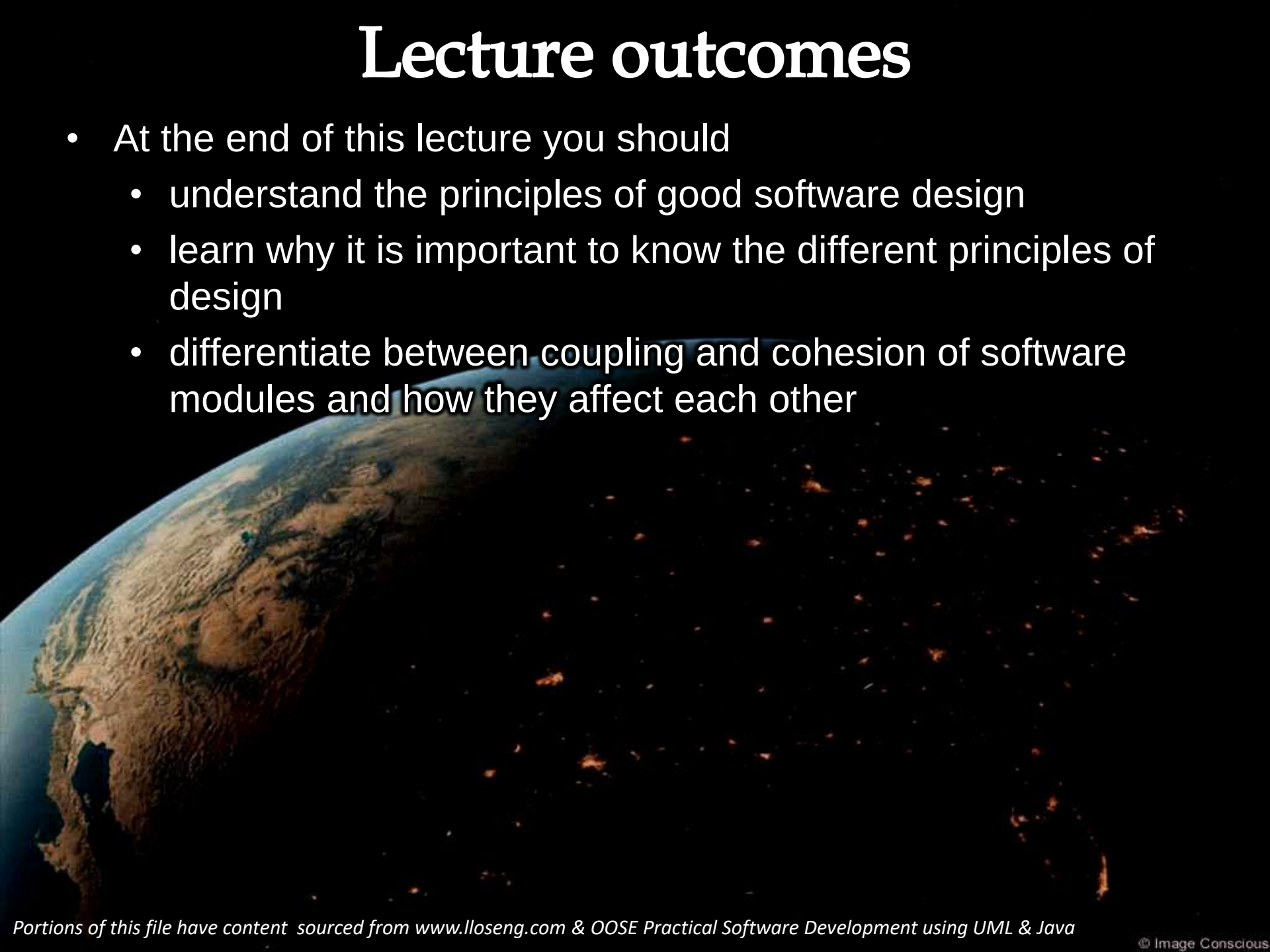

OO System Design 2
TCP2201
Object-oriented Analysis and Design
© Image Conscious

Lecture outcomes

- At the end of this lecture you should
  - understand the principles of good software design
  - learn why it is important to know the different principles of design
  - differentiate between coupling and cohesion of software modules and how they affect each other



Portions of this file have content sourced from www.lloseng.com & OOSE Practical Software Development using UML & Java

11 Principles of design

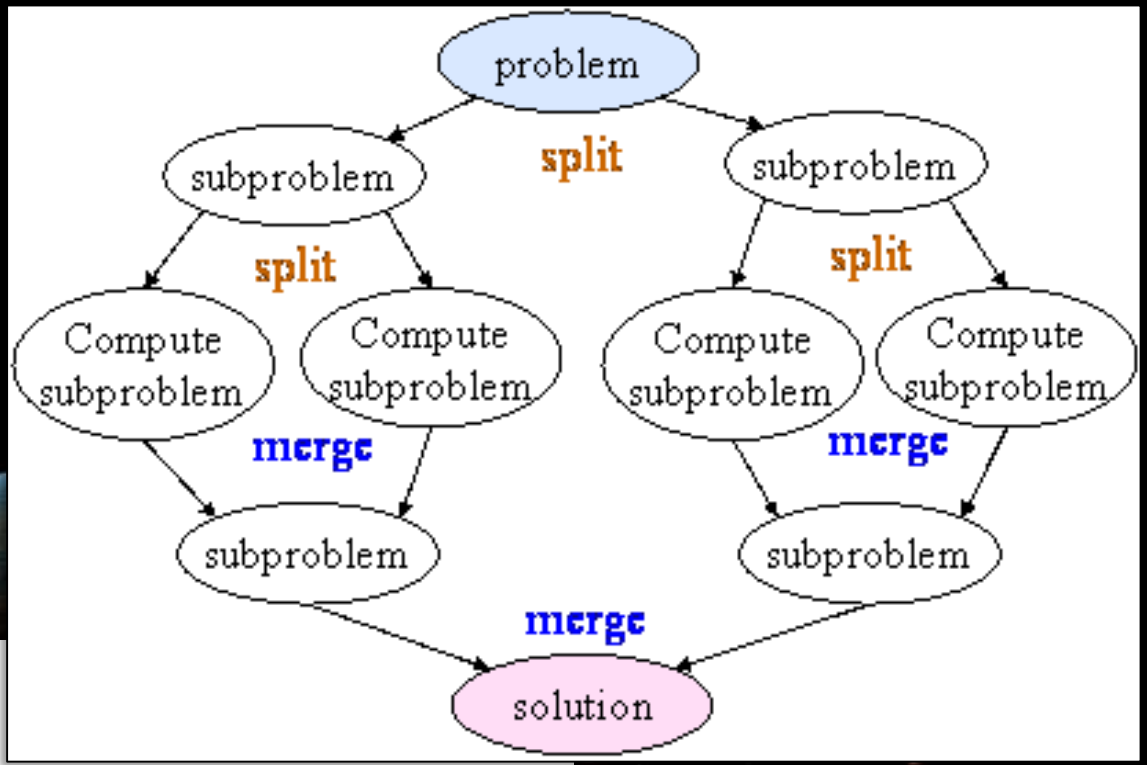
11 principles were devised to promote better software design

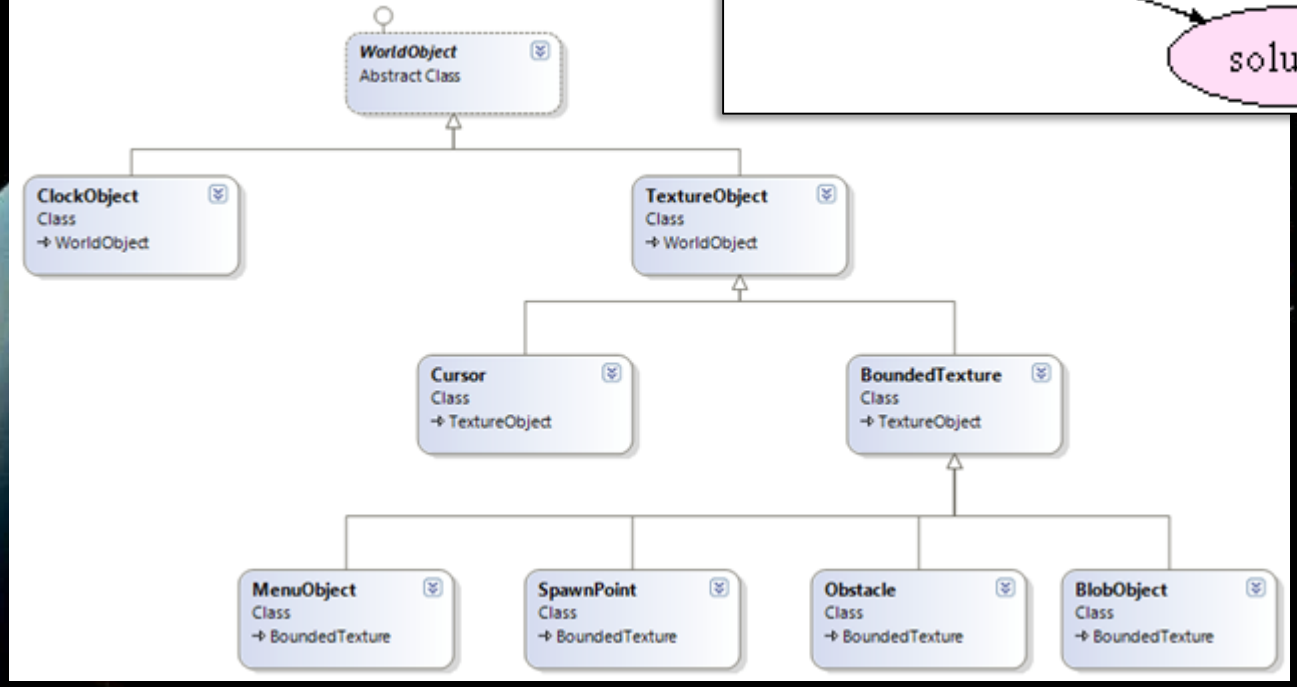
1. Divide and conquer
2. Keep level of abstraction as high as possible
3. Increase reusability
4. Reuse existing designs and code
5. Accommodate design for flexibility
6. Anticipate obsolescence
7. Design for portability
8. Design for testability
9. Design defensively
10. Promote cohesion
11. Reduce coupling

Design Principle 1: Divide and conquer

- Trying to deal with something big all at once is normally much harder than dealing with a series of smaller things
- Software projects are usually handled by teams of developers - separate people can work on each part → An individual software engineer can specialize on sections.
- Each individual component is smaller, and therefore easier to understand.
- Parts can be replaced or changed without having to replace or extensively change other parts.
- Ways of dividing a software system
  - A distributed system is divided up into clients and servers
  - A system is divided up into subsystems
  - A subsystem can be divided up into one or more packages
  - A package is divided up into classes
  - A class is divided up into methods

Procedural breakdown →





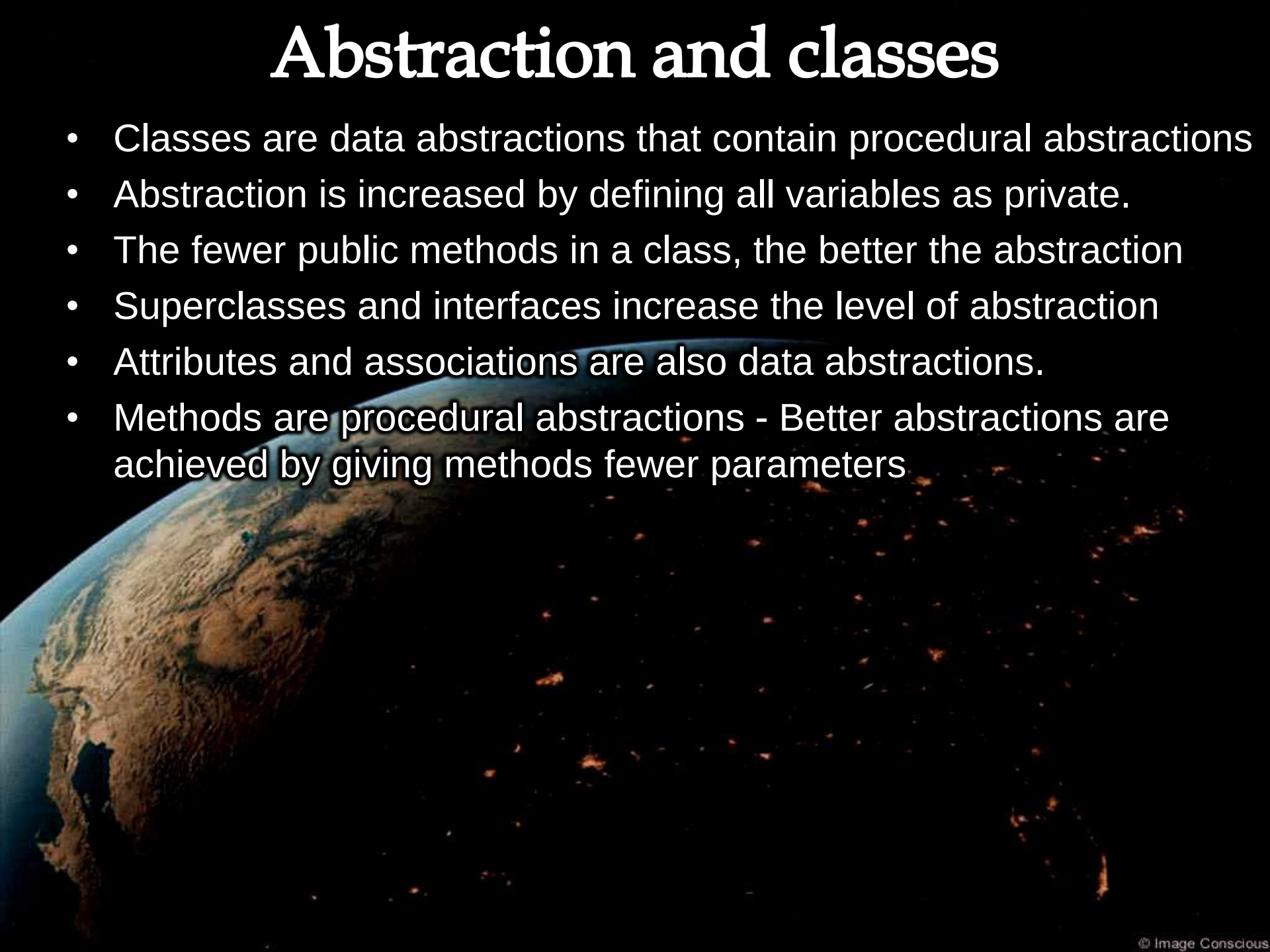
← OOP breakdown

Design Principle 2 : High abstraction

- Keep the level of abstraction as high as possible
- Ensure that your designs allow you to hide or defer consideration of details, thus reducing complexity
- A good abstraction is said to provide information hiding
- Abstractions allow you to understand the essence of a subsystem without having to know unnecessary details

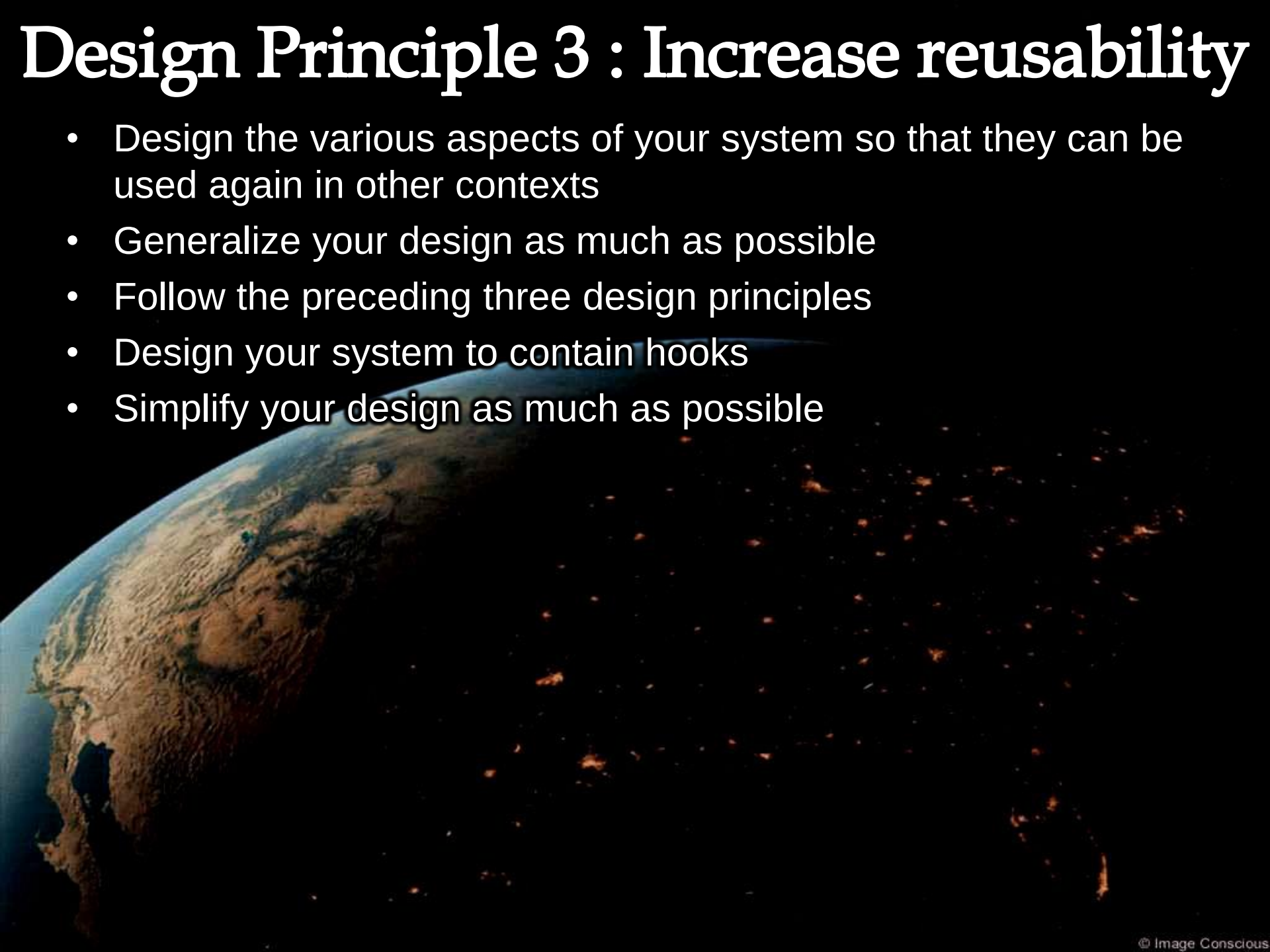
Abstraction and classes

- Classes are data abstractions that contain procedural abstractions
- Abstraction is increased by defining all variables as private.
- The fewer public methods in a class, the better the abstraction
- Superclasses and interfaces increase the level of abstraction
- Attributes and associations are also data abstractions.
- Methods are procedural abstractions - Better abstractions are achieved by giving methods fewer parameters



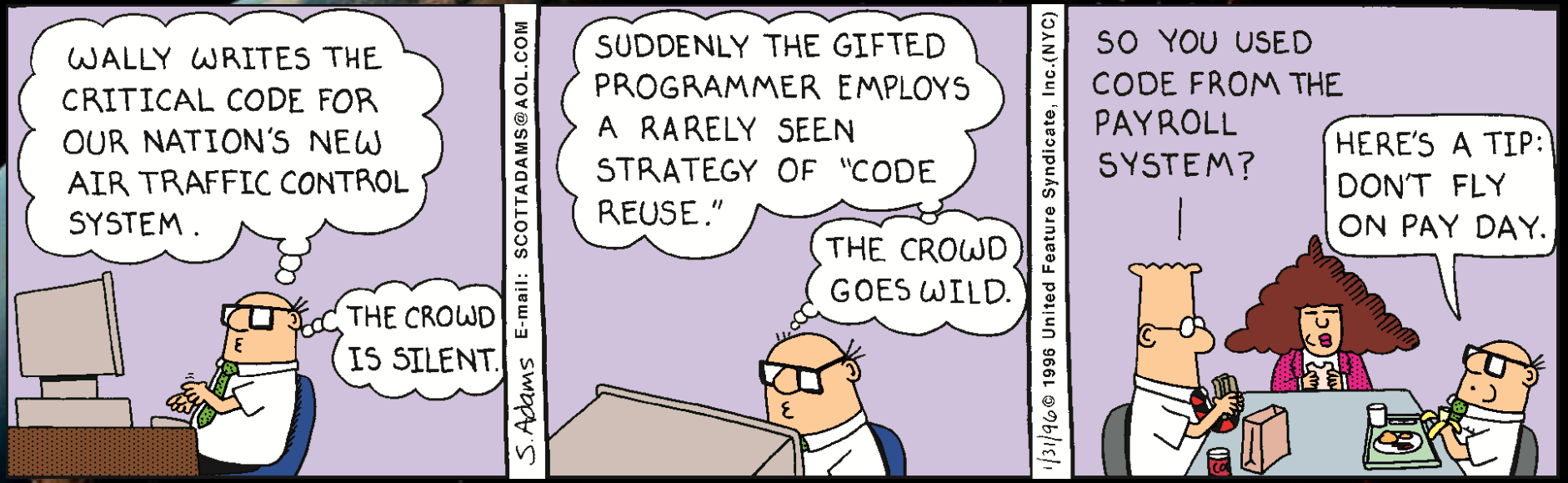
Design Principle 3 : Increase reusability

- Design the various aspects of your system so that they can be used again in other contexts
- Generalize your design as much as possible
- Follow the preceding three design principles
- Design your system to contain hooks
- Simplify your design as much as possible



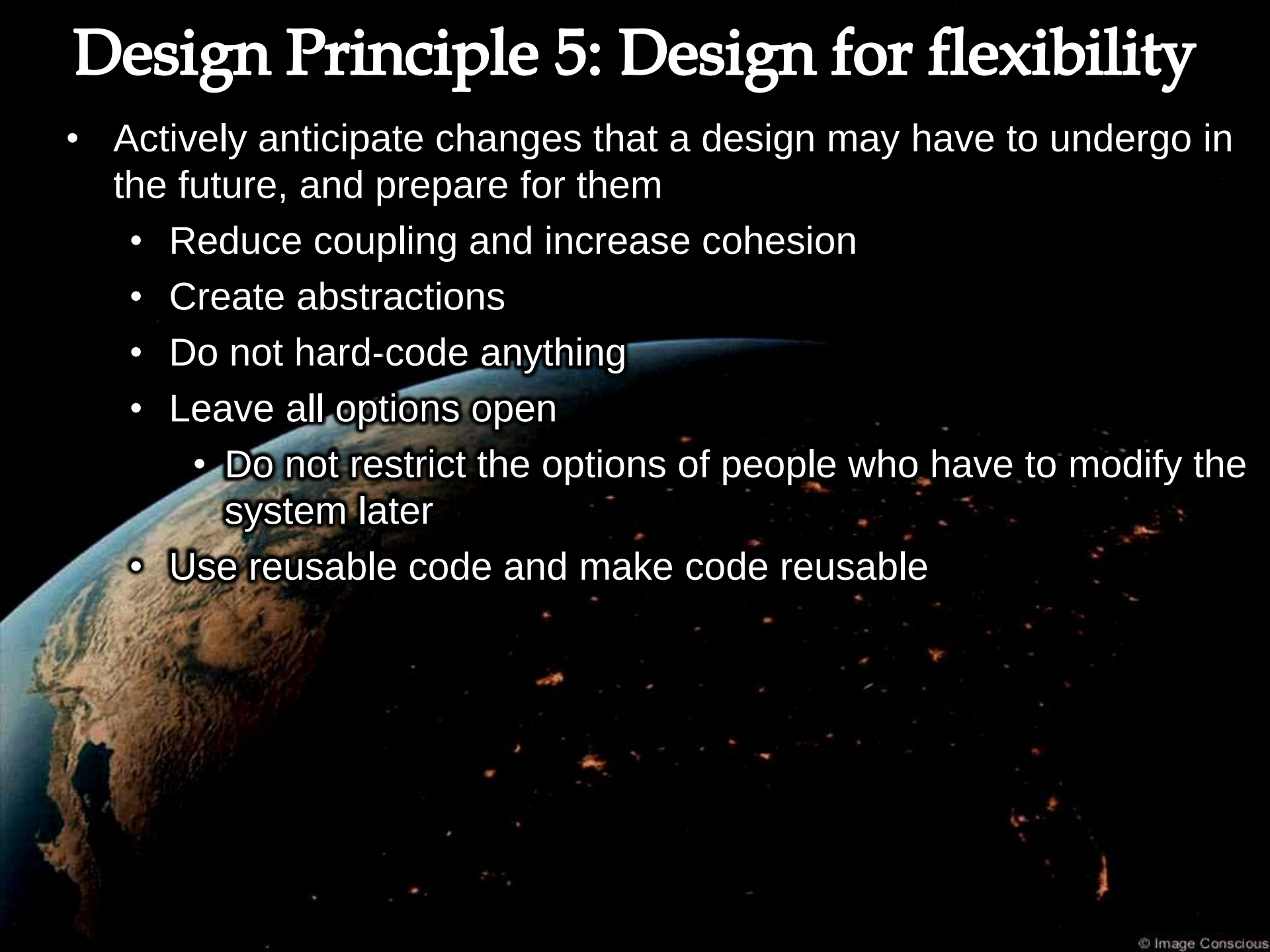
Design Principle 4: Reuse existing designs

- Design with reuse is complementary to design for reusability
  - Actively reusing designs or code allows you to take advantage of the investment you or others have made in reusable components
    - *Cloning* should not be seen as a form of reuse
  - Reuse should not be abused



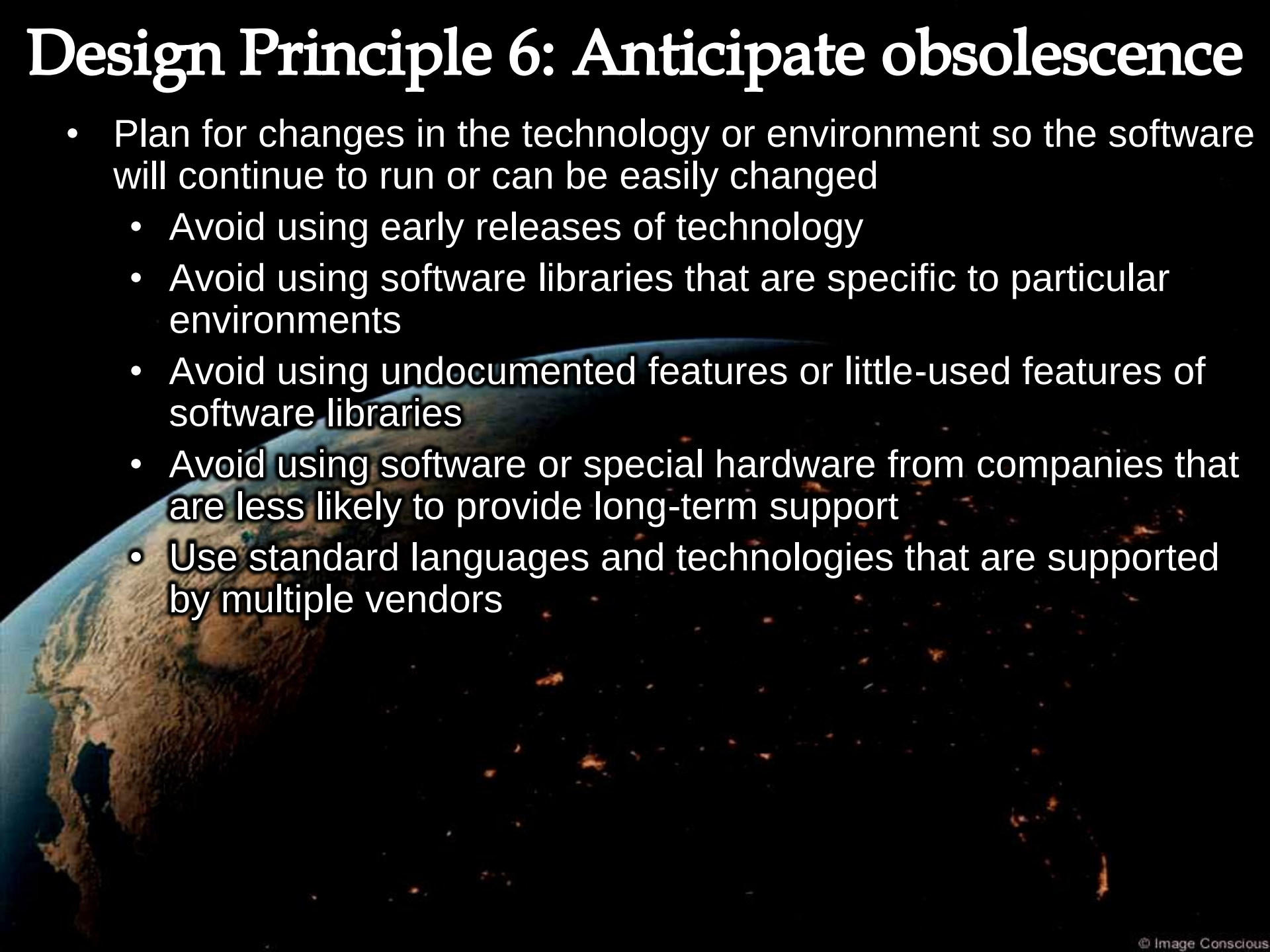
Design Principle 5: Design for flexibility

- Actively anticipate changes that a design may have to undergo in the future, and prepare for them
  - Reduce coupling and increase cohesion
  - Create abstractions
  - Do not hard-code anything
  - Leave all options open
    - Do not restrict the options of people who have to modify the system later
  - Use reusable code and make code reusable



Design Principle 6: Anticipate obsolescence

- Plan for changes in the technology or environment so the software will continue to run or can be easily changed
  - Avoid using early releases of technology
  - Avoid using software libraries that are specific to particular environments
  - Avoid using undocumented features or little-used features of software libraries
  - Avoid using software or special hardware from companies that are less likely to provide long-term support
  - Use standard languages and technologies that are supported by multiple vendors



Design Principle 7: Design for Portability

- Have the software run on as many platforms as possible
  - Avoid the use of facilities that are specific to one particular environment
  - E.g. a library only available in Microsoft Windows
  - Maybe difficult depending on the programming language used

Design Principle 8: Design for Testability

- Take steps to make testing easier
  - Design a program to automatically test the software
    - Ensure that all the functionality of the code can by driven by an external program, bypassing a graphical user interface
  - In Java, you can create a `main()` method in each class in order to exercise the other methods not used in the main program

Design Principle 9: Design defensively

- Never trust how others will try to use a component you are designing
- Handle all cases where other code might attempt to use your component inappropriately
- Check that all of the inputs to your component are valid: the preconditions
- Unfortunately, over-zealous defensive design can result in unnecessarily repetitive checking

Design Principle 10: Increase cohesion

- A subsystem or module has high cohesion if it keeps together things that are related to each other, and keeps out other things that are irrelevant
- This makes the system as a whole easier to understand and change
  - Types of cohesion:
    - Functional
    - Layer
    - Communicational
    - Sequential
    - Procedural
    - Temporal
    - Coincidental

- There are other cohesion types not discussed including logical, utility and object.

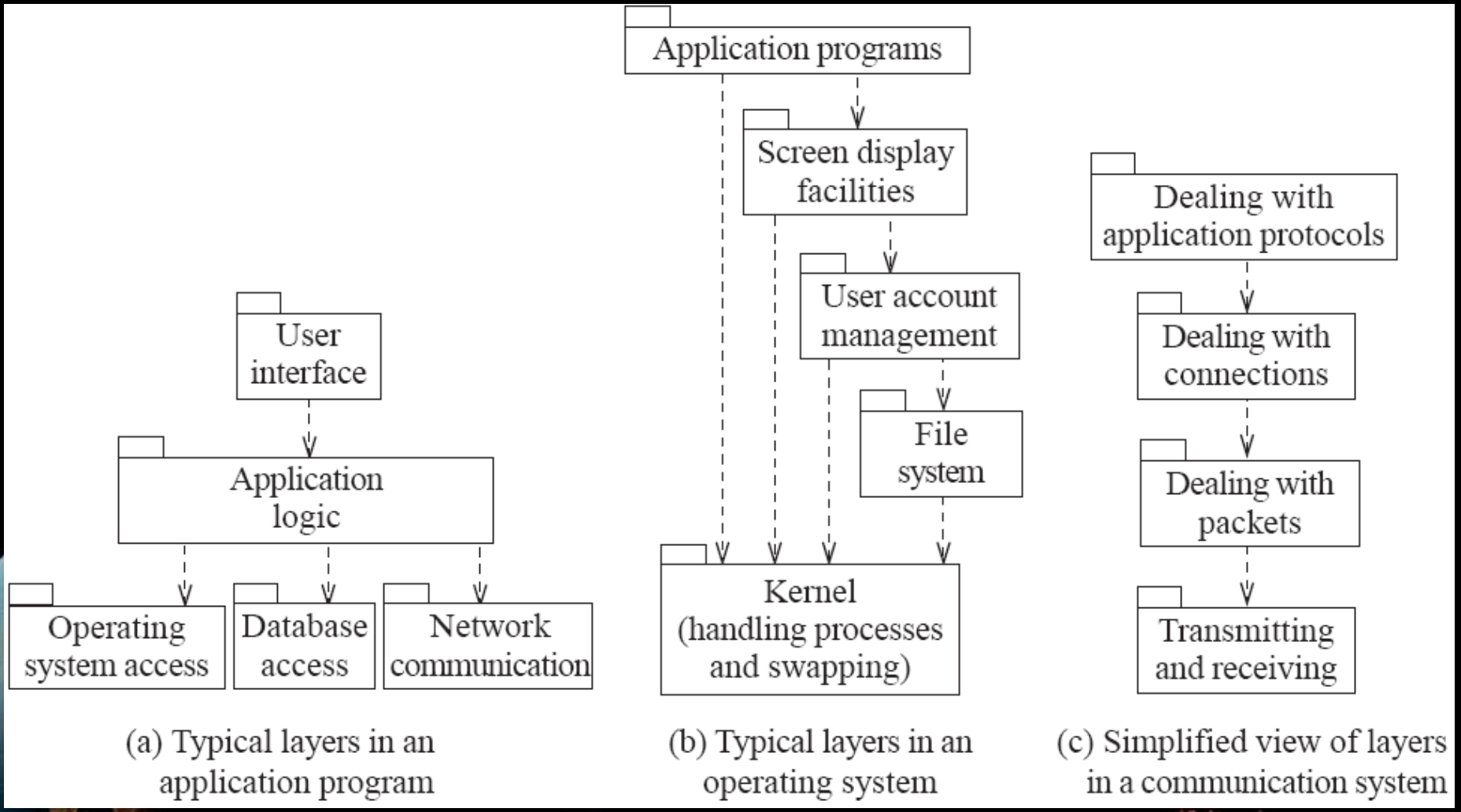
Functional cohesion

- This is achieved when all the code that computes a particular result is kept together - and everything else is kept out i.e. when a module only performs a single computation, and returns a result, without having side-effects.
- Modules that update a database, create a new file or interact with the user are not functionally cohesive → require multiple independent sub tasks to complete
- Functional cohesive module examples include:
  - algorithm to sum elements in array, logic to validate user credentials, method to convert °C to °F, etc.

Advantages	Disadvantages
• Easier to understand • More reusable • Easier to replace	• May result in over-simplification of modules • End design has hundreds (or more) small modules

Layer cohesion

- All the facilities for providing or accessing a set of related services are kept together, and everything else is kept out
- The layers can form a hierarchy where
  - Higher layers can access services of lower layers and lower layers do not access higher layers
  - There are circumstances where access is bi-directional
- The set of procedures through which a layer provides its services is the application programming interface (API)
- You can replace a layer without having any impact on the other layers - You just replicate the API
- Similar advantages/disadvantages to functional cohesion



(a) Typical layers in an application program

(b) Typical layers in an operating system

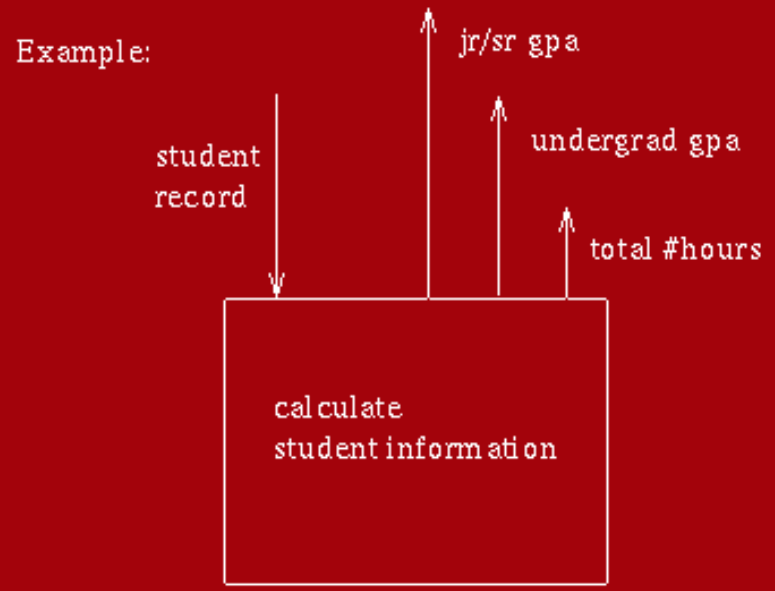
(c) Simplified view of layers in a communication system

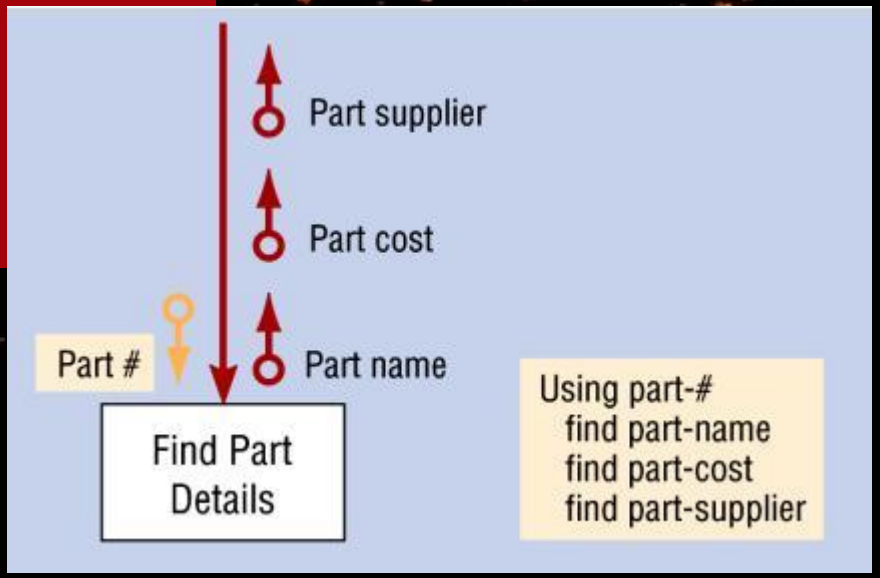
Communicational cohesion

- All the modules that access or manipulate certain data are kept together (e.g. in the same class) - and everything else is kept out
    - A class would have good communicational cohesion if
        - all the system's facilities for storing and manipulating its data are contained in this class and
        - the class does not do anything other than manage its data.
- Main advantage: When you need to make changes to the data, you find all the code in one place
- Disadvantage: Other modules making use of the communication module may only require part of the input/output data generated. The designer must either filter/discard unneeded data (creating added work) *or* create another communication module which causes duplication of code making maintenance difficult.

communicational

A communicationally cohesive module is one whose elements perform differnt functions, but each function references the same input information or output.

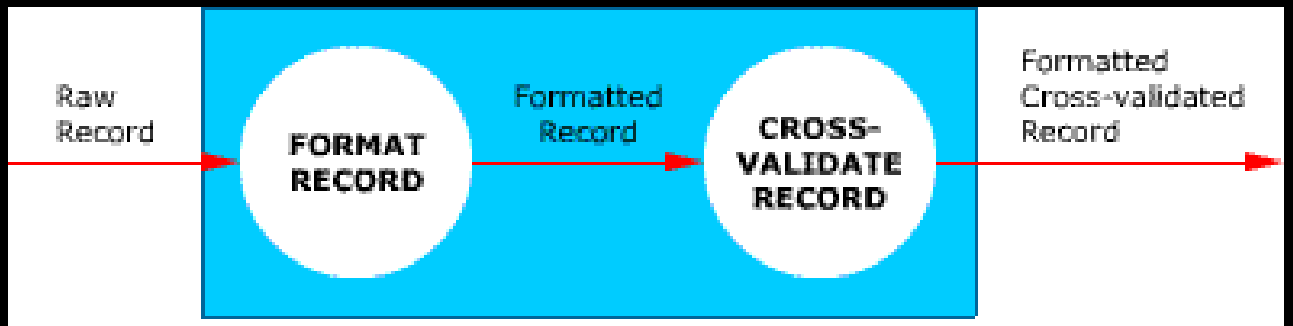




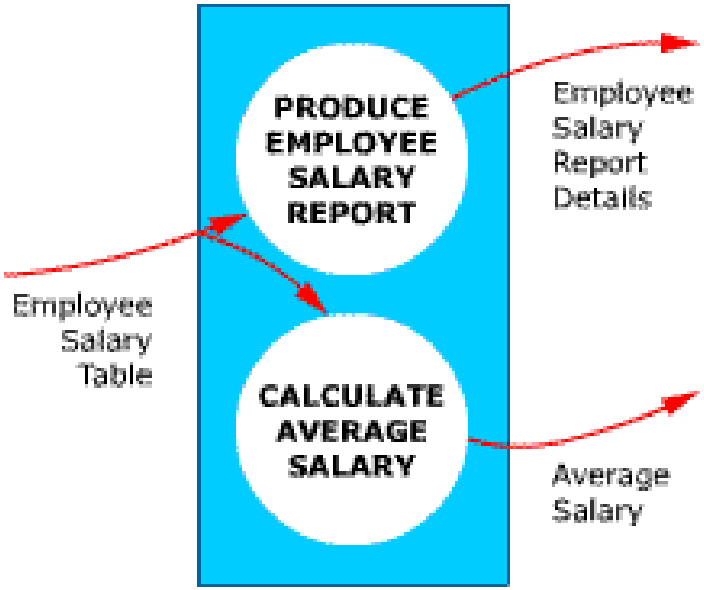
Example from David Stotts Dept. of Computer Science, University of North Carolina

Sequential cohesion

- Sequential procedures, in which one procedure provides input to the next, are kept together – and everything else is kept out
- You should achieve sequential cohesion, only once you have already achieved the preceding types of cohesion
- Closely follows and related to the S.O.P. as written in requirements documents
- Main advantage: a sequentially cohesive module is easy to maintain and has good coupling.
- Disadvantage: does not promote re-use of modules because the sequence of the module generally only meets the requirements for that particular module.



FORMAT AND CROSS-VALIDATE RECORD
Module Is Sequentially Cohesive



PRODUCE EMPLOYEE SALARY REPORT
AND CALCULATE AVERAGE SALARY
Module Is Communicationally Cohesive

Procedural cohesion

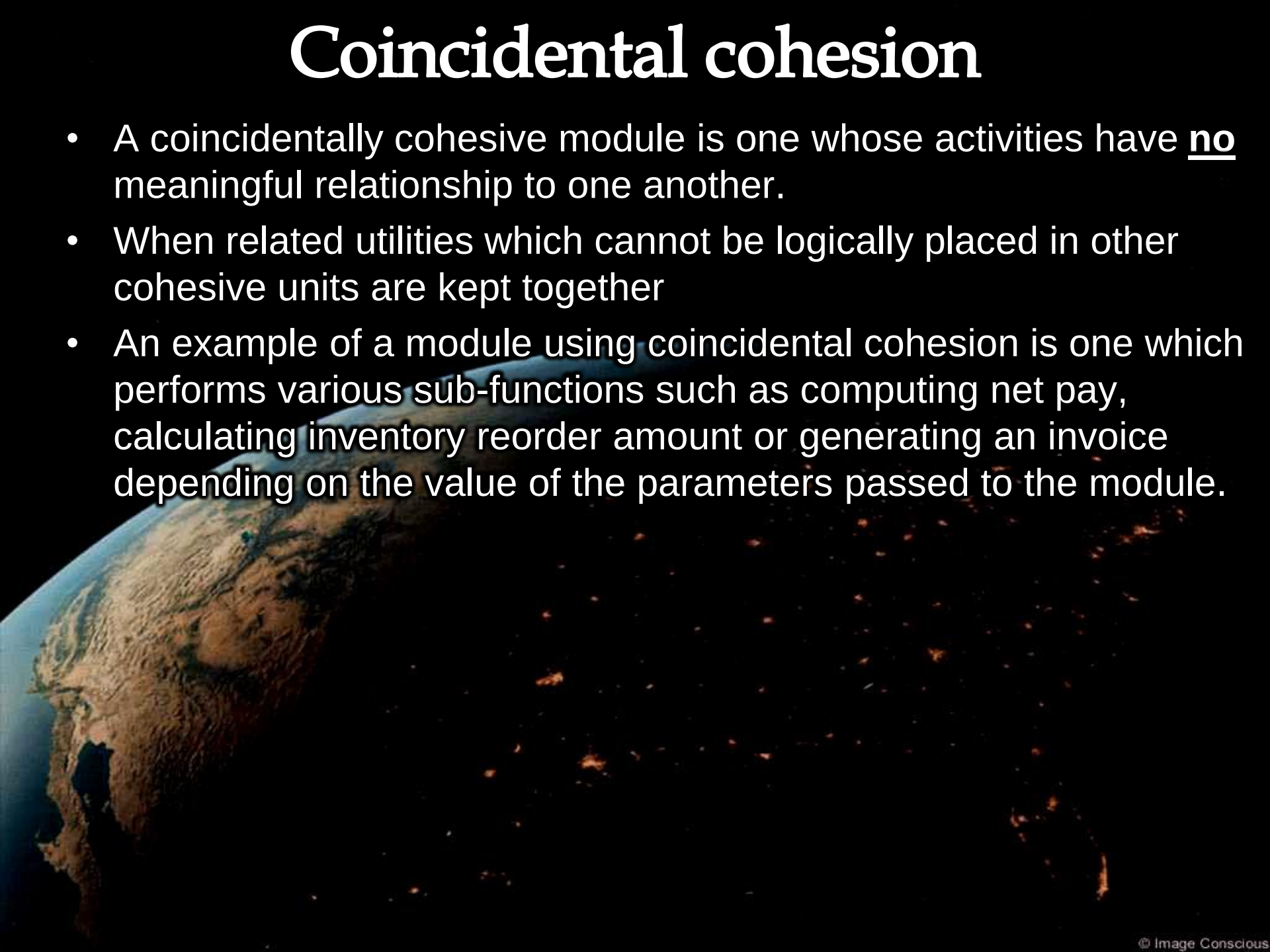
- A procedurally cohesive module performs several different and possibly unrelated activities in which control flows from each activity within the module to the next
- It is also possible that one activity does not necessarily provide input to the next.
- Activities in a procedurally cohesive module are related by possible flow of execution rather than by one problem-related function → often contain a group of functions which make up part of a larger function, but as a group itself performs no real function
- Weaker than sequential cohesion.
- Advantages: similar to sequential cohesion but less obvious sequence
- Disadvantages:
  - modules are usually not reusable
  - not easily maintained as each function may exist independently and changes may/not affect others.

Temporal cohesion

- A temporally cohesive module is one which performs several activities that are related in time
- Operations that are performed during the same phase of the execution of the program are kept together, and everything else is kept out
- For example, placing together the code used during system start-up or initialization.
- Weaker than procedural cohesion.
- Disadvantages:
  - difficult to maintain → any changes to one portion in module will affect how the entire module works.
  - difficult to reuse due ot the nature/order of how the activities execute.

Coincidental cohesion

- A coincidentally cohesive module is one whose activities have **no** meaningful relationship to one another.
- When related utilities which cannot be logically placed in other cohesive units are kept together
- An example of a module using coincidental cohesion is one which performs various sub-functions such as computing net pay, calculating inventory reorder amount or generating an invoice depending on the value of the parameters passed to the module.



Cohesion example

Function A	
Function B	Function C
Function D	Function E

Coincidental
Parts unrelated

Time t_0
Time t_0 + X
Time t_0 + 2X

Temporal
Related by time

Function A part 1
Function A part 2
Function A part 3

Functional
Sequential with complete, related functions

Cohesion example

COINCIDENTAL

All of the following in a single module

- code to check account
- code to change password
- code to generate student report
- code to generate student list
- etc

TEMPORAL

Related to time they need to be done. E.g. launch of application, program must

- initialize array for user id and password
- initialize web session cookie
- encrypt session with generated random key
- prepare greeting for user login

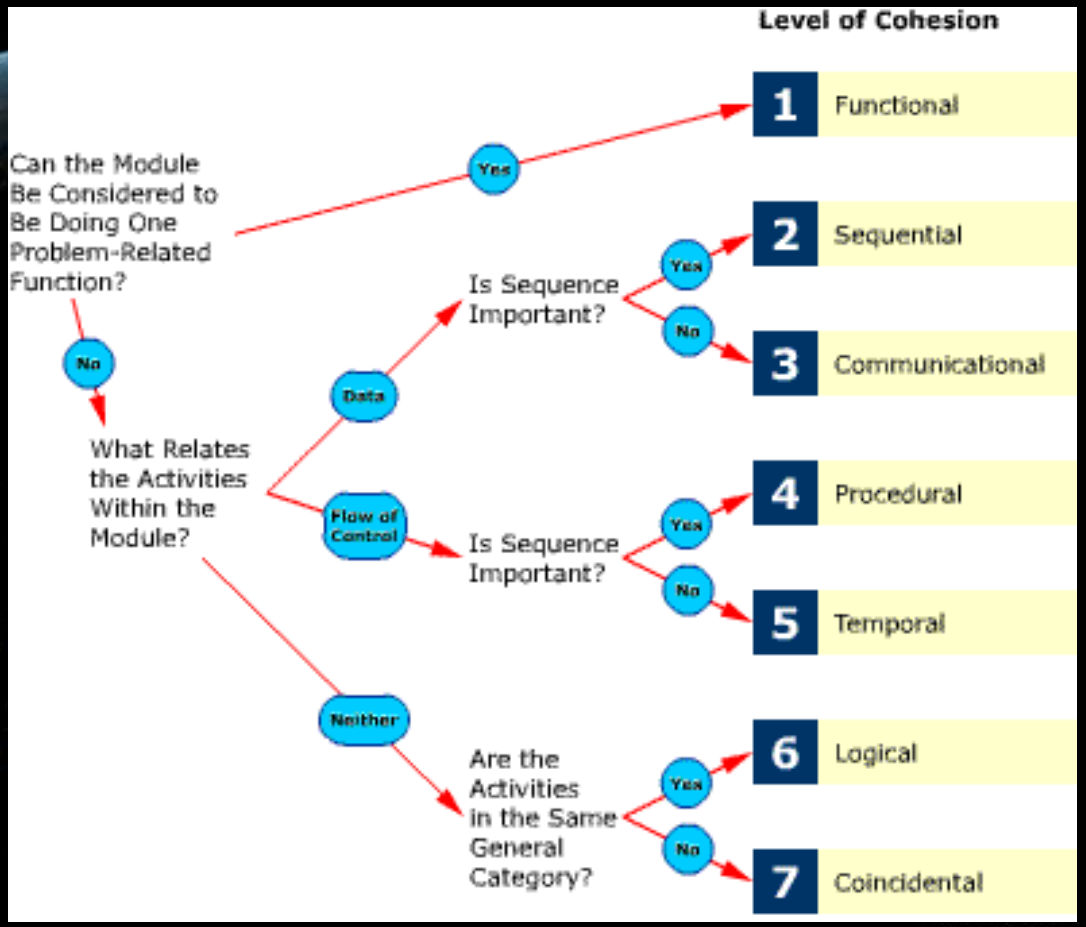
FUNCTIONAL

Each of the following is separate module

- logic to display login date
- logic to prompt for user id
- logic to validate user id
- logic to load user menu
- etc.

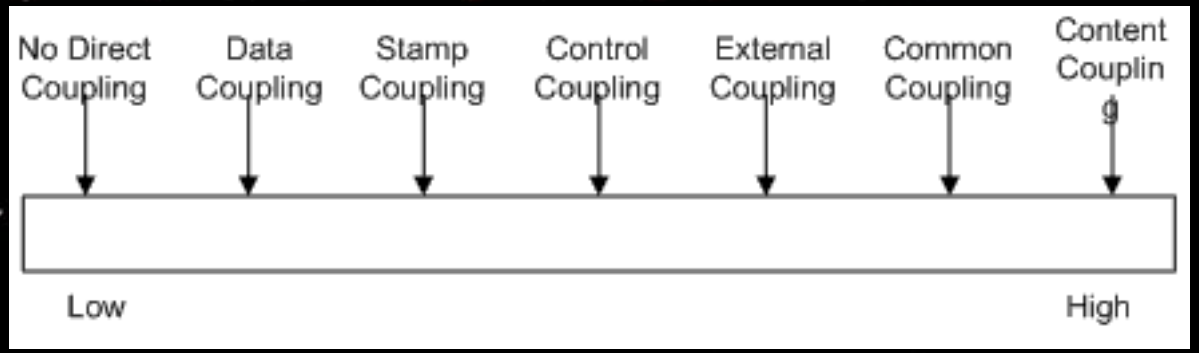
How much cohesion?

- Functional cohesion should be achieved where ever possible.
- Communicational and sequential cohesion represent a satisfactory fallback level.
- Logical, temporal, and procedural cohesion should be reserved for application specific, non-reusable code.
- Coincidental cohesion should not be used at all.

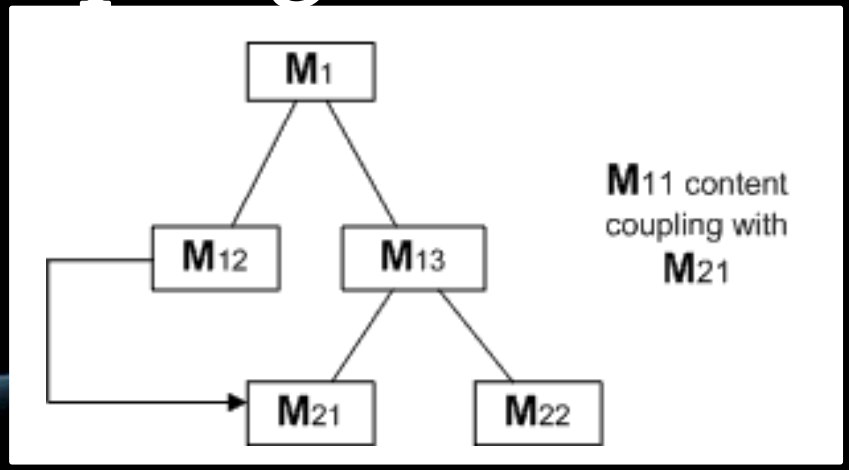


Design Principle 11: Reduce coupling

- Coupling occurs when there are interdependencies between one module and another
- When interdependencies exist, changes in one place will require changes somewhere else.
- A network of interdependencies makes it hard to see at a glance how some component works.
- Types of coupling:
  - Content
  - Control
  - Data
  - Type use
  - External
  - Common
  - Stamp
  - Routine Call
  - Inclusion/Import



Content coupling



- Occurs when one component surreptitiously modifies data that is internal to another component
- This means objects are free to access *another* object's data and once that object's methods are written, it will be coupled to the content of the other object – hence, content coupling
- To reduce content coupling you should
  - Encapsulate all instance variables → declare them private and provide get and set methods
  - Use the immutable or read only interface design patterns
- A worse form of content coupling occurs when you directly modify an instance variable of an instance variable

Example

```
public class Animal;
{
   String species;
   ...
   void setSpecies(String spec){...}
}

public class Zoo
{
   Animal zebra = new Animal();
   ...
   zebra.species = "Equus quagga"}
   ...
}
```

An external object is allowed to manipulate another object's properties

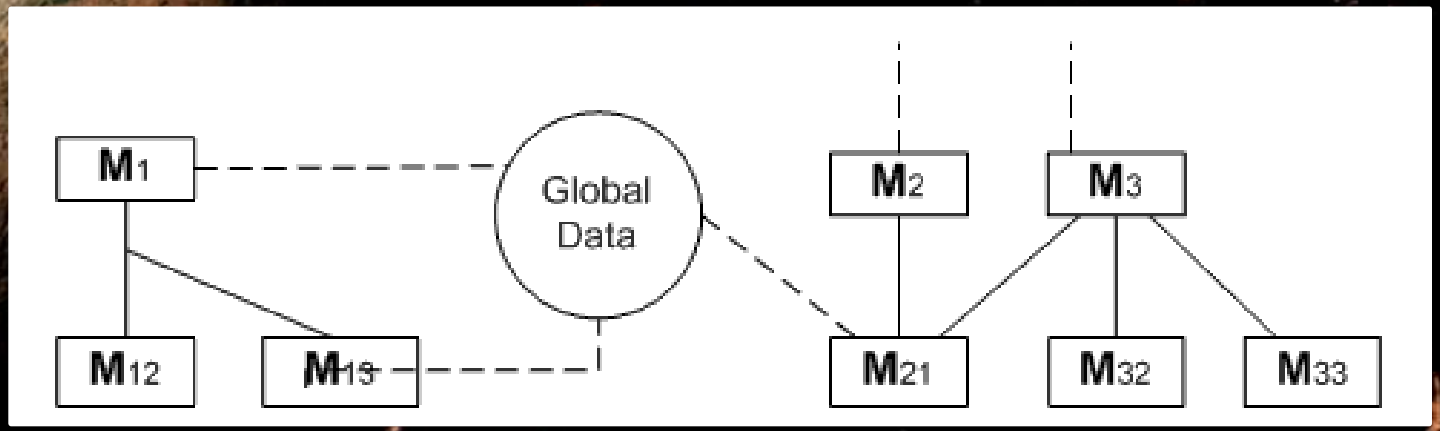
Content coupling

```
public class Line
{
  private Point start, end;
  ...
  public Point getStart() { return start; }
  public Point getEnd()  { return end; }
}

public class Arch
{
  private Line baseline;
  ...
  void slant(int newY)
  {
    Point theEnd = baseline.getEnd();
    theEnd.setLocation(theEnd.getX(),newY);
  }
}
```

Proper way of accessing data is through the interface of the object

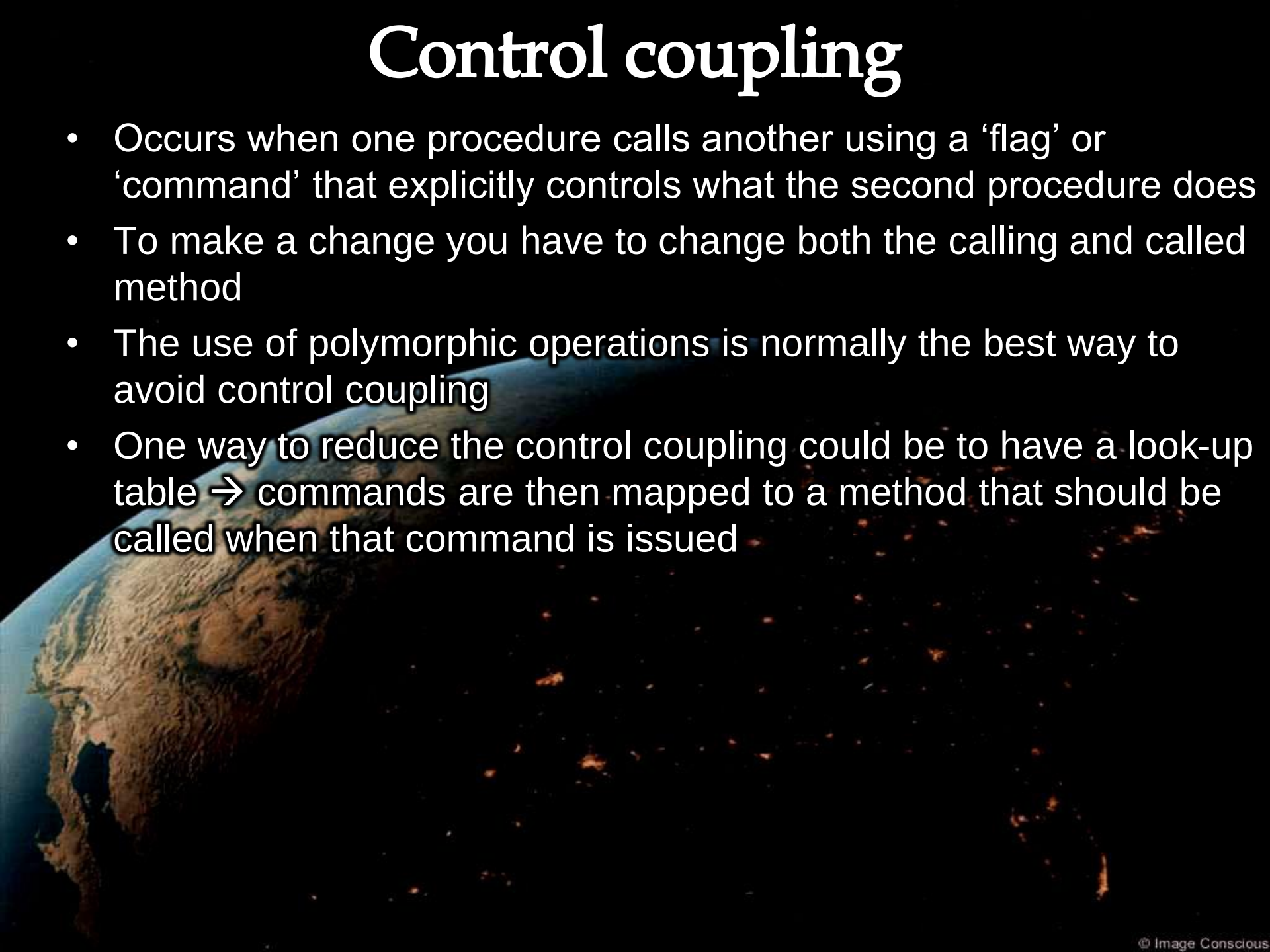
Common coupling

- Occurs whenever you use a global variable - all the components using the global variable become coupled to each other
- A weaker form of common coupling is when a variable can be accessed by a subset of the system's classes e.g. in a Java package
- Can be acceptable for creating global variables that represent system-wide default values
- The Singleton pattern provides encapsulated global access to an object



Control coupling

- Occurs when one procedure calls another using a ‘flag’ or ‘command’ that explicitly controls what the second procedure does
- To make a change you have to change both the calling and called method
- The use of polymorphic operations is normally the best way to avoid control coupling
- One way to reduce the control coupling could be to have a look-up table → commands are then mapped to a method that should be called when that command is issued



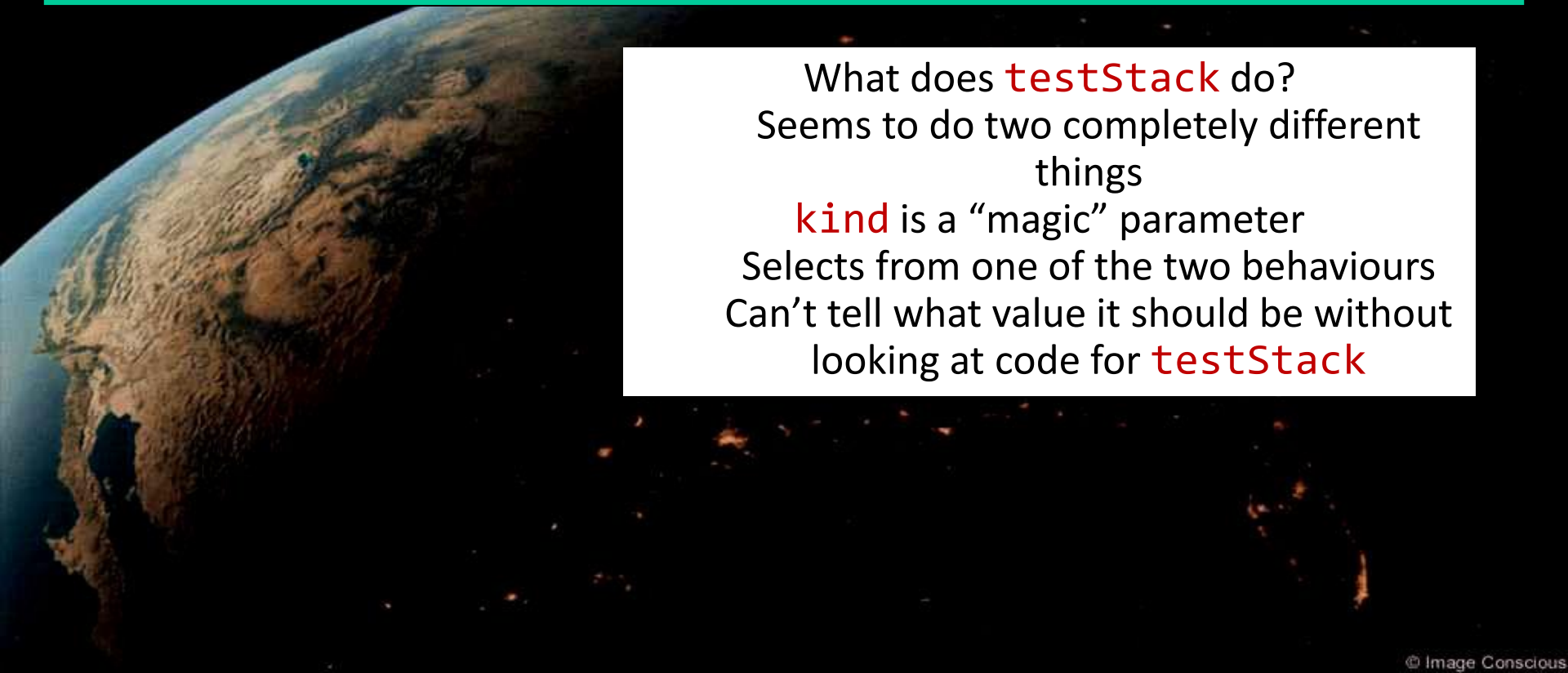
Example

```
public routineX(String command)
{
    if (command.equals("drawCircle")
    {
        drawCircle();
    }
    else
    {
        drawRectangle();
    }
}
```

Notice that to draw a rectangle, the caller must send anything else *other* than *drawCircle* instead of calling drawRectangle directly.

Control coupling example

```
int testStack(int kind, Stack S) {
     if ( ((kind==1)&&(S.num == 0)) || ((kind==2)&&(S.num==MAX)) )
         return 1;
     else return 0;
}
```



What does `testStack` do?
Seems to do two completely different things
`kind` is a “magic” parameter
Selects from one of the two behaviours
Can’t tell what value it should be without looking at code for `testStack`

- Previous code worked fine (if called correctly) but is better to have two functions and name them intuitively

```
int emptyStack (Stack S) {
     if (S.num == 0)
          return 1;
     else return 0;
}

int fullStack (Stack S) {
     if (S.num == MAXSTACK)
          return 1;
     else return 0;
}
```

- It is clear from the function names what the functions do
- Don’t have to guess at values of parameters
- No longer are the functions relying on a “control” function (i.e. no longer coupled through the control)

Another control coupling example

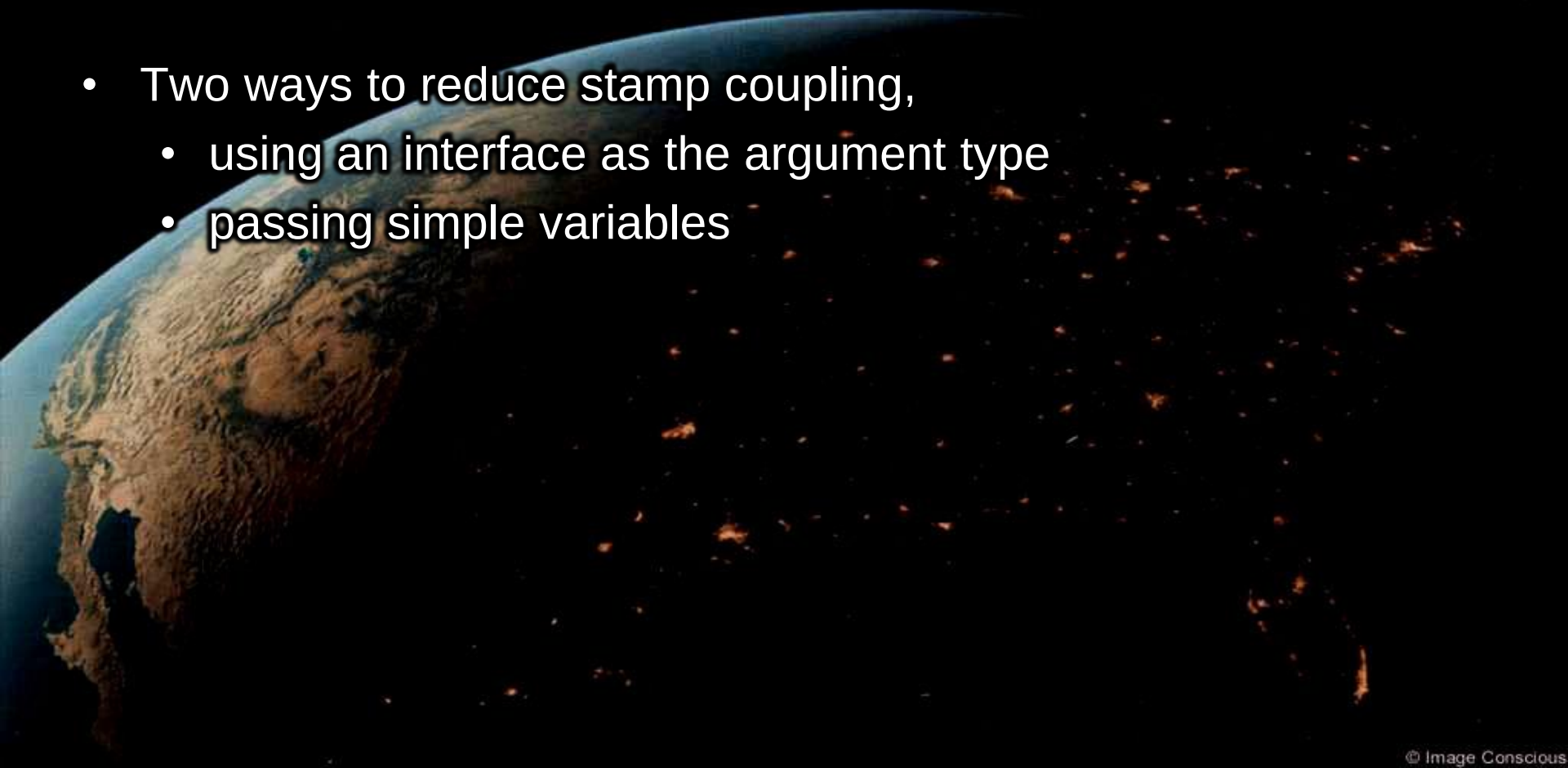
```
void printReport (int which)  {
     switch (which) {
          case 1:  /*print monthly financial report */
          case 2:  /*print report on Web usage */
          // etc …
     }
}
```

- Person coding a function which calls printReport has to remember number, or go check printReport code
- Easier to remember function name than code number
- To avoid: Break up functions doing two different things into individual ones with descriptive names

Stamp coupling

- Occurs whenever one of your application classes is declared as the type of a method argument
- Since one class now uses the other, changing the system becomes harder → Reusing one class requires reusing the other

- Two ways to reduce stamp coupling,
  - using an interface as the argument type
  - passing simple variables



Example

```
public class Emailer
{
  public void sendEmail(Employee e, String text)
  {...}
  ...
}
```

Any changes to Employee access methods will force this class to change too

Avoiding stamp coupling

- Use simple data types instead of class objects

```
public class Emailer
{
  public void sendEmail(String name,String email,String text)
  {...}
}
```

- Or use interface objects in arguments to avoid

```
public interface Addressee{
  public abstract String getName();
  public abstract String getEmail();
}
public class Employee implements Addressee {…}
public class Emailer {
  public void sendEmail(Addressee e, String text)
  {...}
}
```

Stamp coupling example

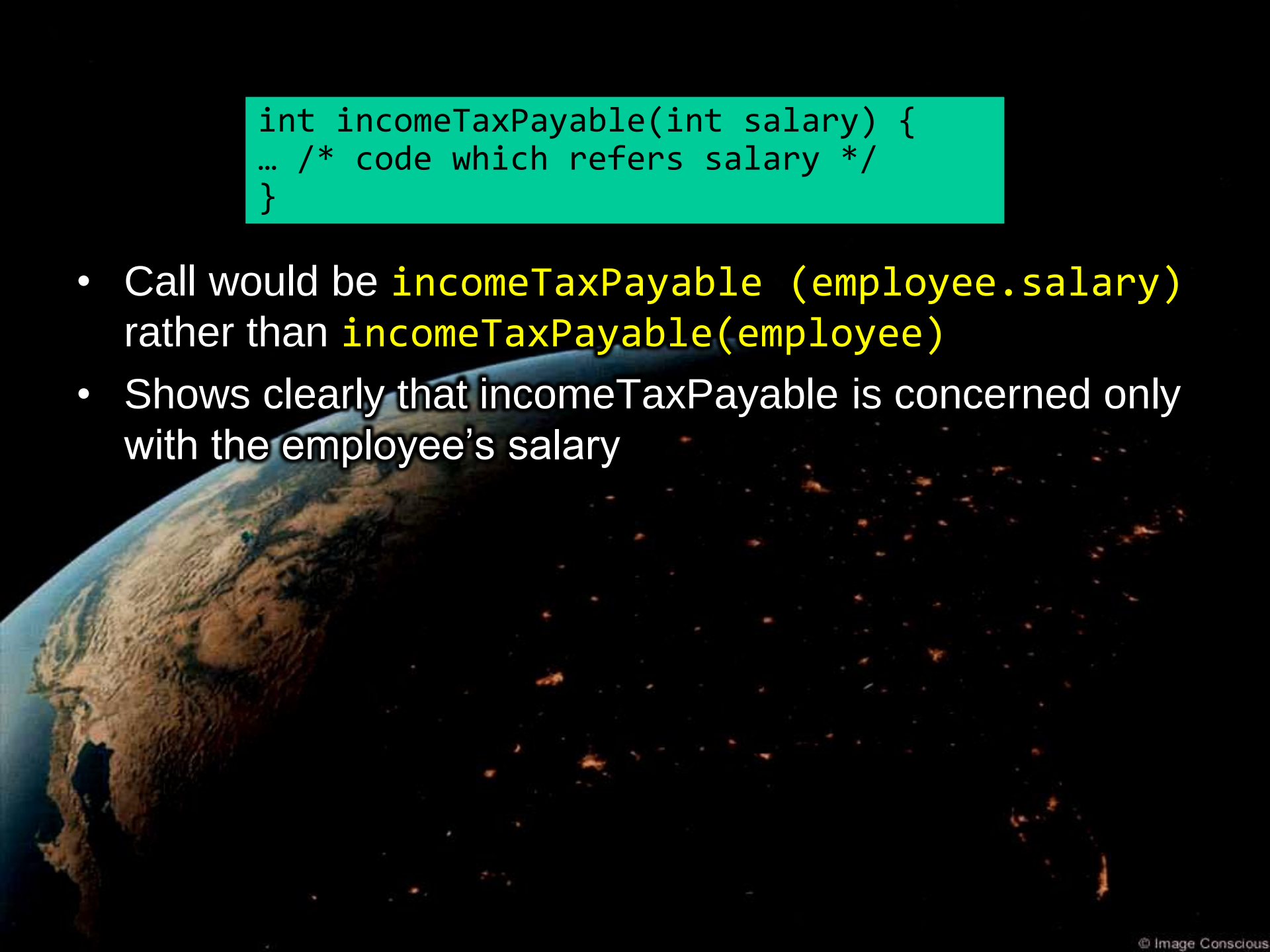
- A function using only a small part of a parameter. Example: code for finding how much income tax to deduct from employee's salary:

```
int incomeTaxPayable(Person p) {
… /* code which refers to only p.salary */
}
```

- This code tells us that the function `incomeTaxPayable` is interested in all the content from the object `Person` – when in reality it only requires the `salary` amount from `Person`
- If the calling function can extract all that is needed for the called function, it should

```
int incomeTaxPayable(int salary) {
… /* code which refers salary */
}
```

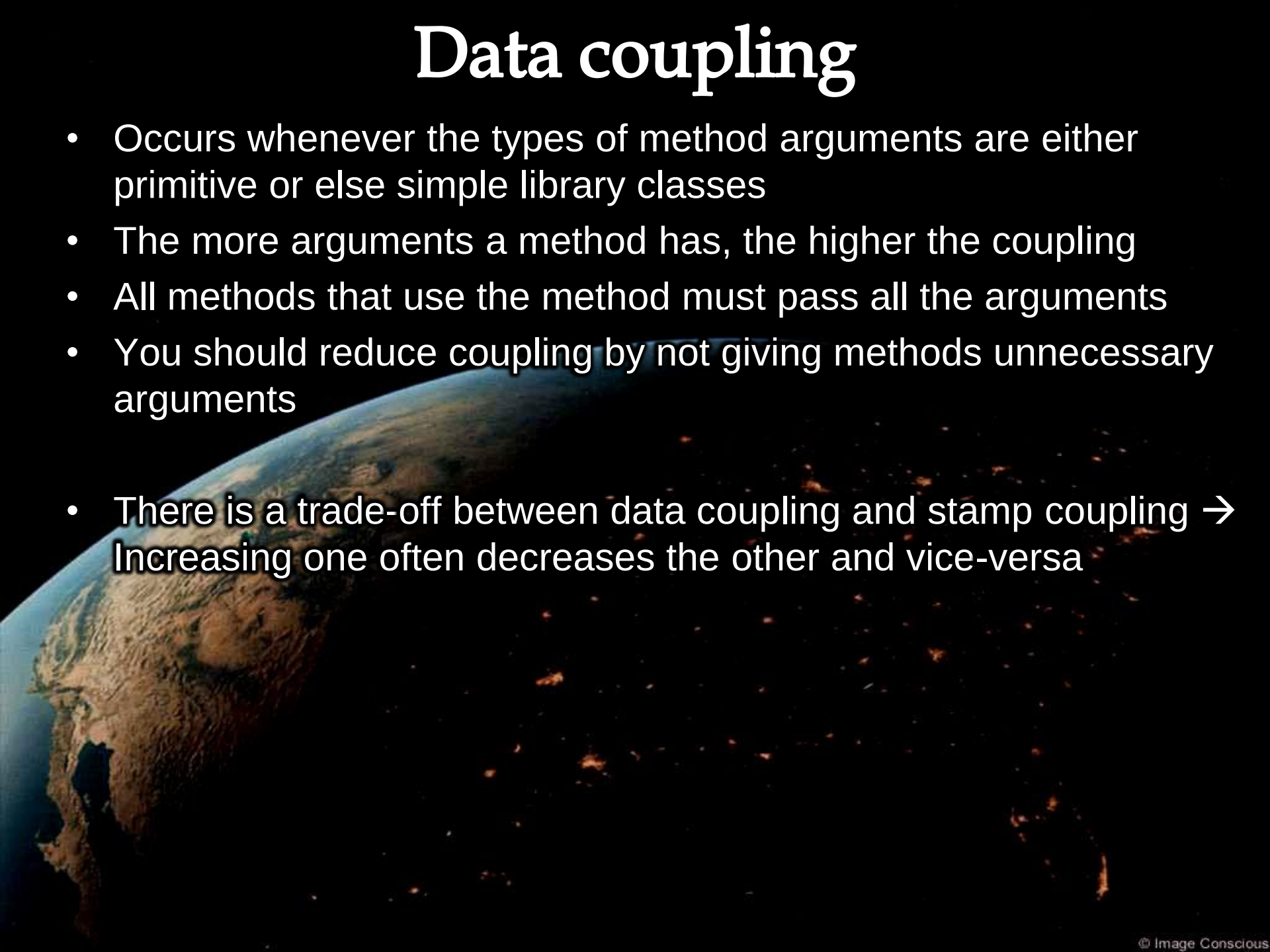
- Call would be `incomeTaxPayable (employee.salary)` rather than `incomeTaxPayable(employee)`
- Shows clearly that incomeTaxPayable is concerned only with the employee's salary



Data coupling

- Occurs whenever the types of method arguments are either primitive or else simple library classes
- The more arguments a method has, the higher the coupling
- All methods that use the method must pass all the arguments
- You should reduce coupling by not giving methods unnecessary arguments

- There is a trade-off between data coupling and stamp coupling → Increasing one often decreases the other and vice-versa

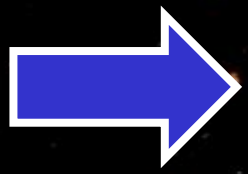


Routine call coupling

- Occurs when one routine (or method in an object oriented system) calls another
- The routines are coupled because they depend on each other's behaviour → Routine call coupling is always present in any system.

- If you repetitively use a sequence of two or more methods to compute something, then you can reduce routine call coupling by writing a single routine that encapsulates the sequence.

```
method foo(){
a();
b();
c();
a();
b();
c();
}
```

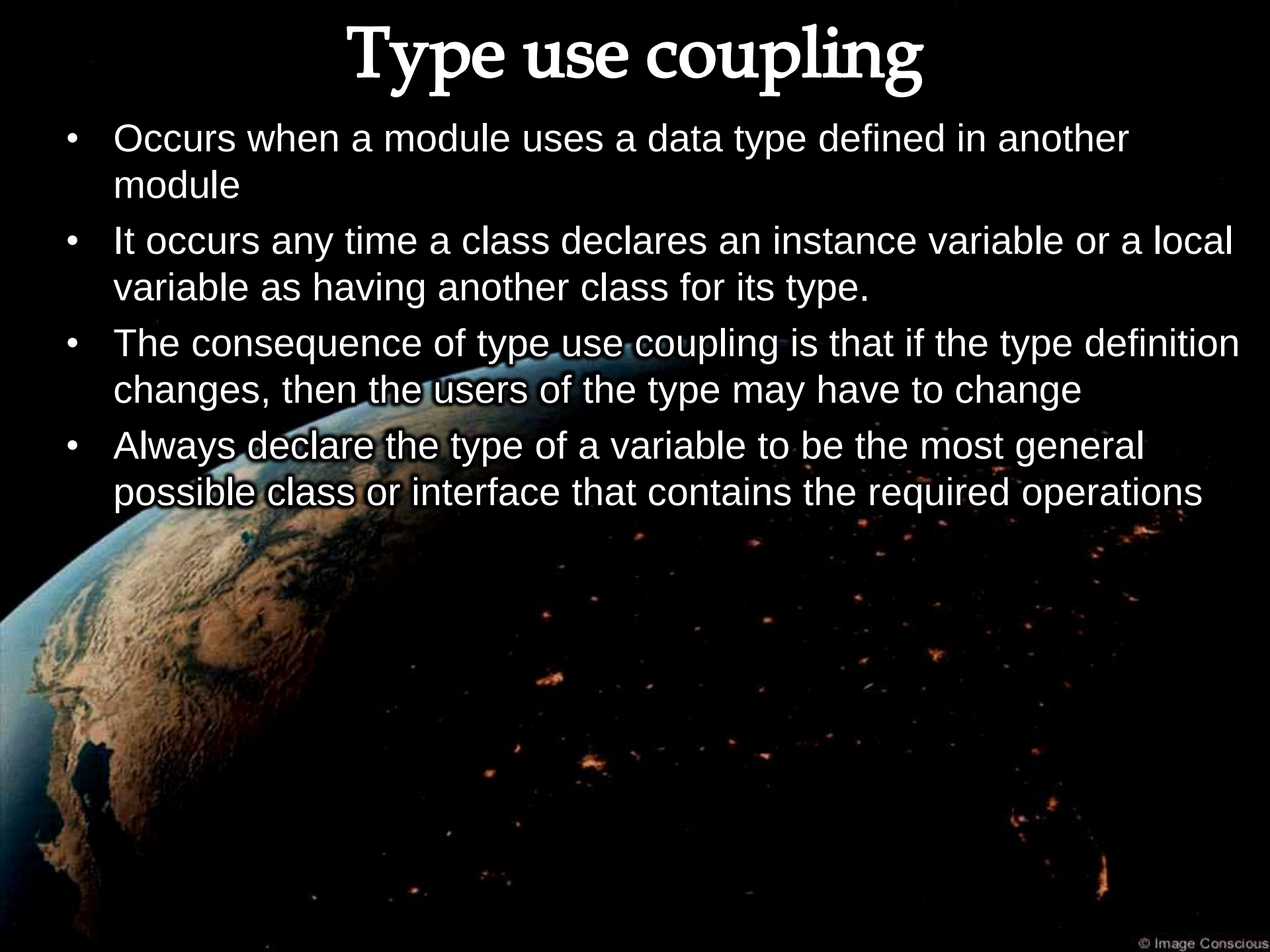


```
method foo(){
abc();
abc();
}
```

```
method abc(){
a();
b();
c();
}
```

Type use coupling

- Occurs when a module uses a data type defined in another module
- It occurs any time a class declares an instance variable or a local variable as having another class for its type.
- The consequence of type use coupling is that if the type definition changes, then the users of the type may have to change
- Always declare the type of a variable to be the most general possible class or interface that contains the required operations



Type Use Coupling

If new datatypes are required, they must be hardcoded

```
class Travel
{
    Car c=new Car();
    void startJourney()
    {
       c.move();
    }
}

class Car
{
  void move()
  {
     // code logic...
  }
}
```

Car is used as a datatype in the Travel class

Any changes to the Car class may change Travel

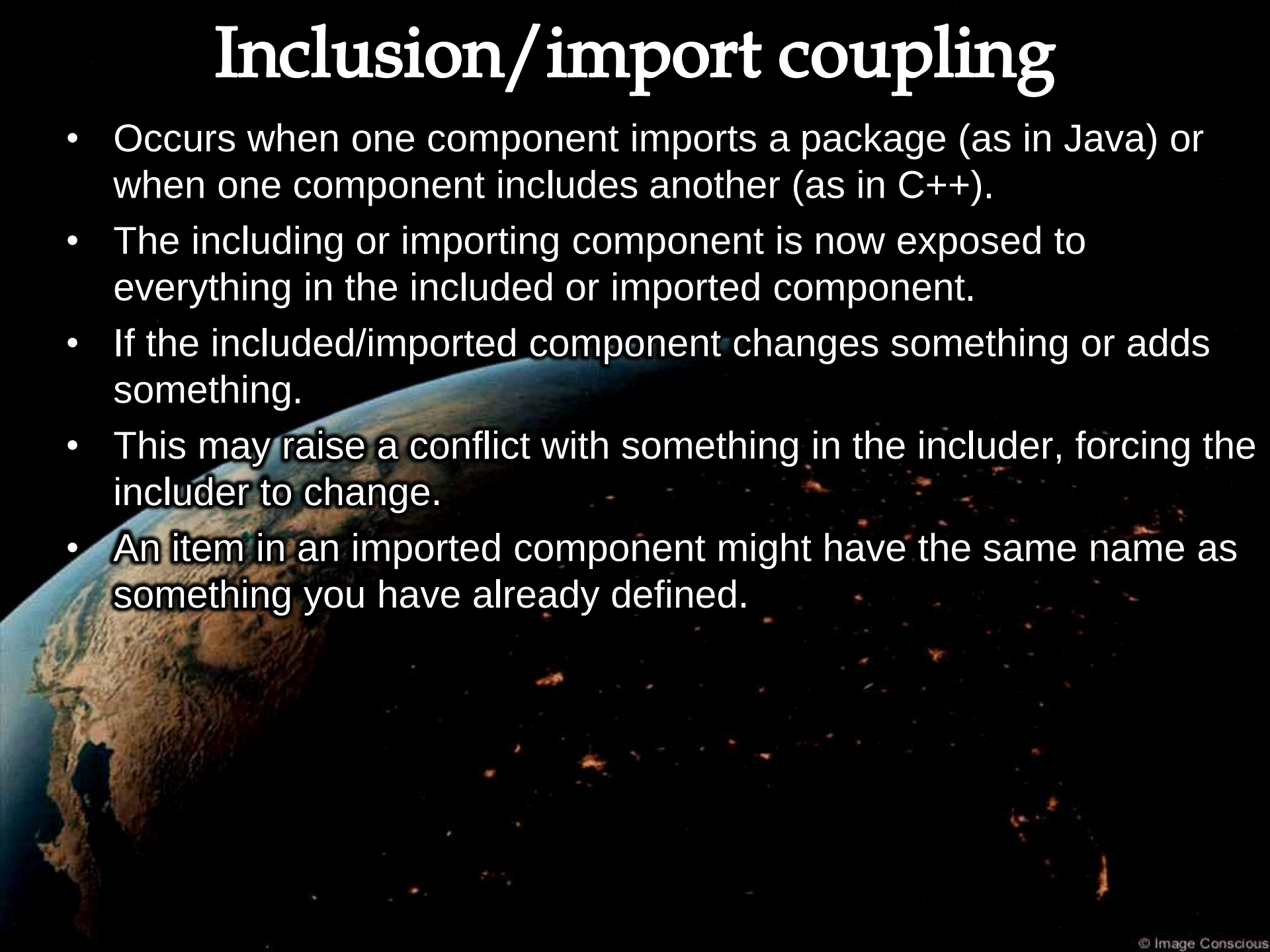
```
class Travel
{
    Vehicle v;
    // something will call setV
    void setV(Vehicle vin){
         this.v = vin;
    }
    void startJourney()
    {
       v.move();
    }
}

interface Vehicle
{
  void abstract move();
}
```

```
class Car implements Vehicle
{
  void move(){
     // code logic
  }
}
```

```
class Bike implements Vehicle
{
  void move(){
     // code logic
  }
}
```

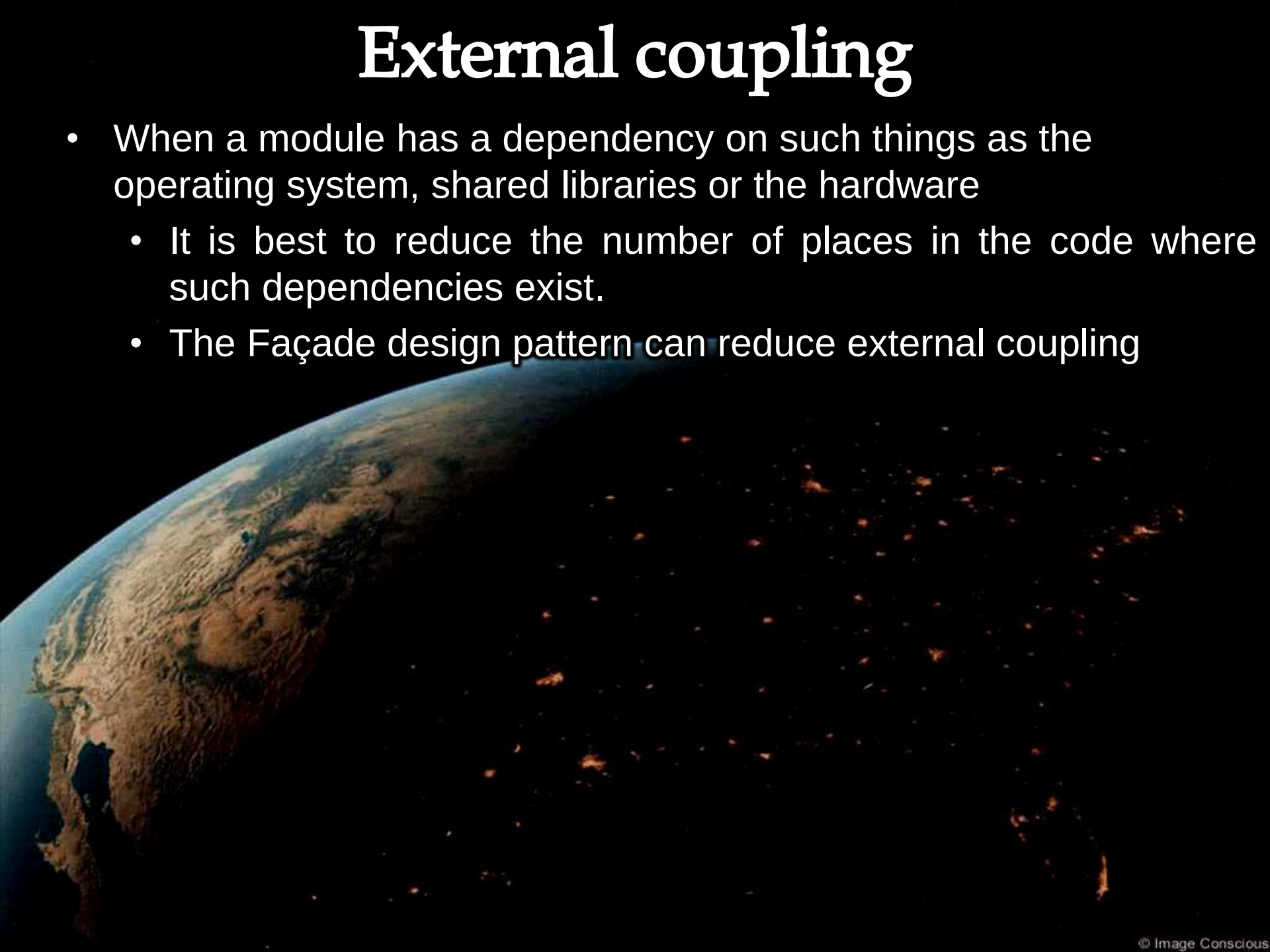
Inclusion/import coupling

- Occurs when one component imports a package (as in Java) or when one component includes another (as in C++).
- The including or importing component is now exposed to everything in the included or imported component.
- If the included/imported component changes something or adds something.
- This may raise a conflict with something in the includer, forcing the includer to change.
- An item in an imported component might have the same name as something you have already defined.

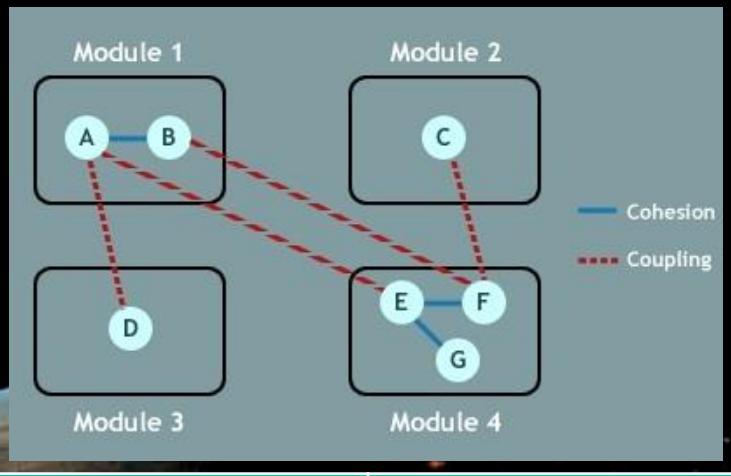


External coupling

- When a module has a dependency on such things as the operating system, shared libraries or the hardware
  - It is best to reduce the number of places in the code where such dependencies exist.
  - The Façade design pattern can reduce external coupling



Coupling vs Cohesion



Cohesion	Coupling
Indication of the relationships WITHIN a module	Indication of the relationships BETWEEN modules
Shows a module's functional strength	Shows the independence between the modules
Levels of cohesion indicate the degree to which a module focuses on a single thing	Levels of coupling indicate the degree to which a module is connected to other modules
OO system design should target HIGH cohesion in modules (i.e. a cohesive module focuses on a single task)	OO system design should target low coupling (i.e. dependency between modules should be at a minimum)

Summary

- Four of the most important principles are: divide up the system so that you can better handle its complexity, increase cohesion so that each component has a clearly identifiable purpose, reduce coupling so that there is minimal dependency between components, and increase abstraction so that you can work with the design without having to understand its details